Proactive Microservice Failure Mitigation using LogBERT Anomaly Detection in Kubernetes

Akella Harshith Saipaavan 160124737166
Bavikadi Mohaseen Hussain 160124737170
M Jaidev 160124737186
Ponna Manish Reddy 160124737192

Department of Information Technology (IT-3), Chaitanya Bharathi Institute of Technology (A), Hyderabad, India

***Abstract*—Microservice applications on Kubernetes fail in ways that liveness probes and threshold alerts detect only after users are affected. We present a closed-loop framework that learns the normal behaviour of a service from its logs and acts before a degradation becomes an outage. Container logs are parsed into template sequences, scored by a LogBERT model trained with masked log-key prediction and a hypersphere objective, and the resulting anomaly score drives a mitigation controller that restarts or scales the affected workload through the Kubernetes API. On a single-node Minikube deployment of the Online Boutique demo application with three injected fault types, the detector reaches an F1-score of 0.91 on held-out fault windows (DeepLog: 0.90). Closed-loop recovery results are pending the experimental runs.**

***Index Terms*—**Kubernetes, microservices, log anomaly detection, LogBERT, self-healing, AIOps, fault injection.

I. INTRODUCTION

Microservice architectures split an application into many small, independently deployed services that communicate over the network [10]. The pay-off is agility and scalability; the price is a large failure surface. A single memory leak, a saturated connection pool or a slow downstream dependency can cascade through the call graph and turn a local fault into a user-visible incident. Kubernetes [6] automates deployment and offers basic recovery, but its built-in mechanisms are fundamentally reactive: a liveness probe restarts a container only after it has already stopped answering, and the Horizontal Pod Autoscaler (HPA) reacts to CPU or memory averages that lag behind the underlying fault.

Logs are the richest and cheapest signal available to detect trouble early. Services emit characteristic message patterns while healthy, and the first symptoms of a fault, such as growing retry messages, garbage-collection warnings or timeouts to a dependency, appear in the log stream well before metrics cross an alerting threshold. Deep-learning detectors such as DeepLog [2] and LogBERT [1] model these sequences and have shown strong results on offline benchmark datasets. Far less work closes the loop by turning a detection into a safe, automatic remedy inside an orchestrator.

This paper makes three contributions: (1) an end-to-end architecture that connects a log pipeline, a LogBERT detector and a mitigation controller around a Kubernetes cluster; (2) a graded policy that maps the suspected fault class and the persistence of an anomaly to a minimal-impact remedy, with a cool-down guard that prevents oscillation; and (3) an empirical study on a Minikube testbed with three injected fault types, comparing detection quality against classical baselines and recovery against the reactive mechanisms built into Kubernetes.

II. RELATED WORK

Log anomaly detection. DeepLog [2] models the log-key sequence with an LSTM and flags a window when the observed key is not among the top-g predictions. LogAnomaly [4] adds semantic template vectors to cope with unseen messages, and LogRobust [5] uses attention to tolerate log instability. LogBERT [1] applies a bidirectional Transformer trained with masked log-key prediction and a volume of hypersphere minimisation objective, capturing context on both sides of a position and outperforming unidirectional models on HDFS, BGL and Thunderbird. Our work reuses this detector but evaluates it on live microservice logs and couples it to remediation.

Draft — reported values are provisional and pending experimental validation

Log parsing. Drain [3] groups messages with a fixed-depth parse tree, runs online and is robust enough for streaming use. We use it to convert raw lines into template identifiers (log keys). The Loghub collection [8] provides reference datasets that guided our window and vocabulary choices.

Self-healing and AIOps. Commercial and open-source orchestrators ship restart and autoscaling rules, but these are threshold-driven. Chaos engineering tools such as Chaos Mesh [7] are used to test resilience rather than to train detectors. Our framework combines both ideas: injected faults produce labelled data for evaluation, and the learned detector triggers the remedy.

III. SYSTEM ARCHITECTURE

Fig. 1 shows the pipeline. Every pod writes to stdout; a collector polls the Kubernetes log API every 2 s and forwards new lines, tagged with pod and service name, to the parser. The parser outputs a stream of (service, timestamp, log key) tuples. A sequencer keeps one sliding window per service. The detector scores each window and publishes a verdict to the mitigation controller, which acts through the Kubernetes API and whose actions are themselves observed through the next windows of logs.

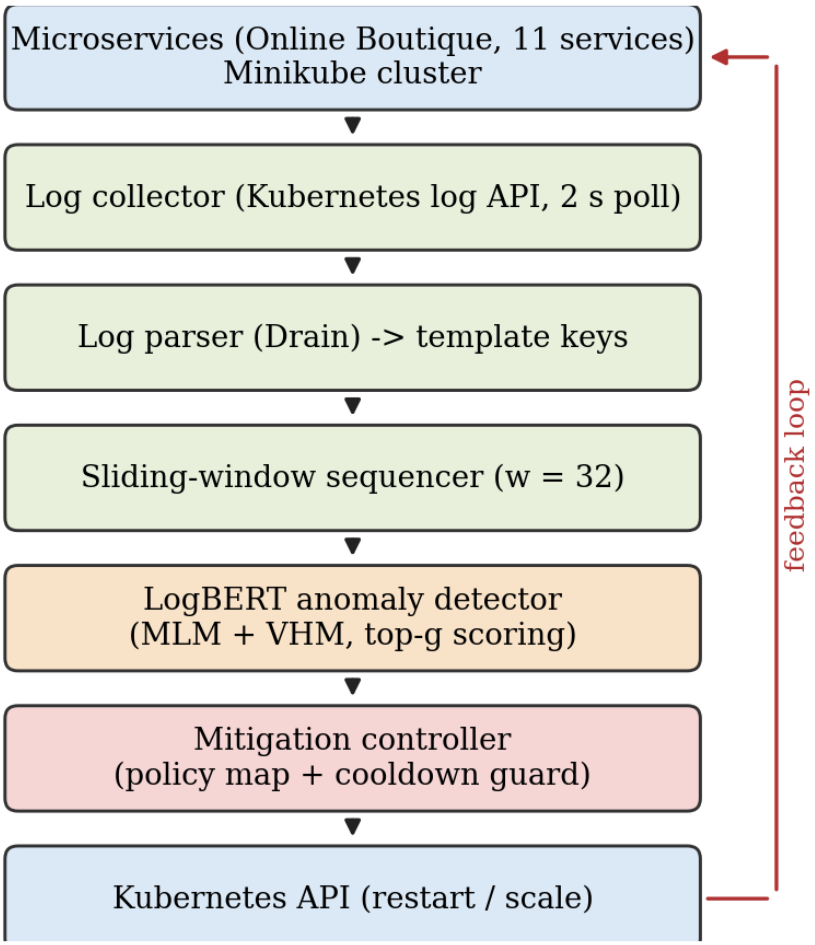


Fig. 1. Architecture of the proactive mitigation framework. Red arrow: the controller's actions change the cluster state, which is observed in subsequent log windows.

A. Log collection and parsing

Structured (JSON) lines are first reduced to their severity, message and HTTP status class. Lines are then masked (IP addresses, UUIDs, hexadecimal identifiers, numbers) and passed through Drain with a similarity threshold of 0.5 and depth 4. Templates are learned from normal traffic only, which yields 48 templates; messages unseen at run time are mapped to a reserved unknown key rather than forcing a retrain. The service name is prefixed to every line, so keys are produced per service and the behaviour of one component is not diluted by the others.

Draft — reported values are provisional and pending experimental validation

B. LogBERT detector

For a window of $w = 32$ consecutive keys, the input embedding is the sum of key, positional and service embeddings. The encoder has 4 layers, 4 attention heads and a hidden size of 256, giving 3.20 M parameters. Training uses two self-supervised objectives on normal data only. The first is masked log-key prediction (MLKP), where 15% of positions are replaced by a mask token and predicted from bidirectional context. The second is volume of hypersphere minimisation (VHM), which pulls the window representation of normal sequences towards a common centre so that deviating sequences land far from it. The total loss is the MLKP loss plus a weighted VHM term with weight 0.1.

At inference, every position of a window is masked exactly once (eight forward passes of four positions each) and we count the positions where the true key is outside the top-g predictions. The anomaly score is that fraction, and a window is anomalous if the score exceeds a threshold *tau*. Both are selected on a validation fault run by maximising F1, giving $g = 3$ and *tau* = 0.03 (Fig. 3). A verdict is raised only after two consecutive anomalous windows to suppress isolated noise.

C. Mitigation controller

The controller turns a verdict into an action using the suspected fault class and the persistence of the anomaly. The fault class is assigned by keyword rules over the raw lines of the anomalous window: timeouts, refused connections and RPC errors mark a dependency fault, and any other service named in those lines is taken as the failing dependency; panics, exceptions and out-of-memory messages mark a crash; an anomalous window with neither is treated as a slow handler. Table I lists the actions. Each service has a cool-down of 60 s between actions, and the controller as a whole is limited to three automatic actions per five minutes; beyond that it only raises an alert. A restart removes one replica at a time.

TABLE I. Mitigation policy (severity-ordered, least disruptive first).

Suspected fault	Severity	Action
Crash / exception burst	Medium	Restart one pod of the service
Dependency error	Medium	Restart one caller pod; scale each named dependency +1 (max 5)
No explicit error (slow handler)	High	Scale replicas by +1 (max 5)
Still anomalous for > 4 windows after an action	Critical	Alert only; stop automatic actions

IV. EXPERIMENTAL SETUP

A. Testbed

Experiments run on one laptop (Intel Core i5, 16 GB RAM, Windows 11) hosting a single-node Minikube 1.39 cluster (Kubernetes v1.37.0) with the Docker driver, limited to 4 vCPU and 6 GB of memory. The cluster runs release v0.10.3 of the Online Boutique demo [9], which consists of 11 services written in several languages and a Redis cache. Load comes from the bundled load generator at its default setting of 10 simulated users. A separate prober pod requests three frontend pages (home, a product page and the cart) in a loop with a 0.5 s pause and a 3 s timeout; its requests define what counts as a user-visible failure. The collector, detector and controller run as processes on the host and reach the cluster through the Kubernetes API; the detector is trained and run on the laptop's GPU (NVIDIA RTX 3050, 6 GB).

B. Dataset

Data are generated in two phases. In the normal phase the application runs under load with no faults; these logs train Drain and all detectors. Normal traffic is not error-free: the load generator itself occasionally triggers a failed checkout or a failed ad lookup, and these lines are kept in the training data. In the fault phase we inject three fault types through the Kubernetes API: *pod kill*, where the pod is force-deleted every 20 s (productcatalogservice, currencyservice, cartservice); *outage*, where the Deployment is scaled to zero replicas (redis-cart,

Draft — reported values are provisional and pending experimental validation

recommendationservice, shippingservice); and *CPU throttling*, where the CPU limit of the running pod is cut in place to 10 millicores (frontend, productcatalogservice, currencyservice). Episodes last about 90 s and follow a shuffled schedule with normal traffic in between. Two fault runs are recorded: one to select thresholds (validation) and one to report results (test). A window is labelled anomalous if it overlaps an injected interval and belongs to the faulted service or to a service that calls it, directly or transitively. Windows in the 60 s after a fault ends are excluded from scoring. Table II summarises the data.

TABLE II. Dataset summary.

Item	Value
Monitored services / templates	10 / 48
Normal log lines	196,999
Training / held-out windows (normal run)	19,635 / 4,918
Validation windows (normal / anomalous)	21,557 / 6,640
Test windows (normal / anomalous)	21,900 / 6,443
Injected fault episodes (validation / test)	18 / 18
Window size / stride	32 keys / 8 keys

C. Baselines and metrics

For detection we compare against a static rule (the share of lines in a window that contain an error keyword), Isolation Forest [11] on template-count vectors, and DeepLog [2]. Every method's threshold is chosen on the validation run and frozen for the test run. For mitigation the baseline is the default Kubernetes behaviour: the probes shipped with Online Boutique plus an HPA targeting 70% CPU on the services that receive the CPU-throttling fault. We report precision, recall and F1 for detection; detection lead time (seconds between the first verdict on an affected service and the first failed probe); mean time to recovery (MTTR, from the first failed probe to the start of the first 10 s stretch without a failed probe); and request availability (the share of successful probes). Each condition is measured in a single run.

The recovery comparison is reported on pod kill and CPU throttling. The outage fault is reported separately: a Deployment scaled to zero cannot be recovered by probes or the HPA until the fault is withdrawn, whereas the controller's scale-up action reverses it directly, so including it in the main comparison would favour the controller by construction.

V. RESULTS AND DISCUSSION

Note: entries marked “pending” have not been measured yet. This draft is provisional until every one of them is replaced with a measured result.

A. Detection quality

Table III reports precision, recall and F1 of each method on the test windows, Fig. 2 breaks F1 down by fault type, and Fig. 3 shows how the LogBERT result depends on the threshold.

TABLE III. Detection results on the test windows.

Method	Precision	Recall	F1
Error-rate rule	1.00	0.82	0.90
Isolation Forest	0.32	0.59	0.42
DeepLog	0.94	0.85	0.90
LogBERT (ours)	**0.97**	**0.86**	**0.91**

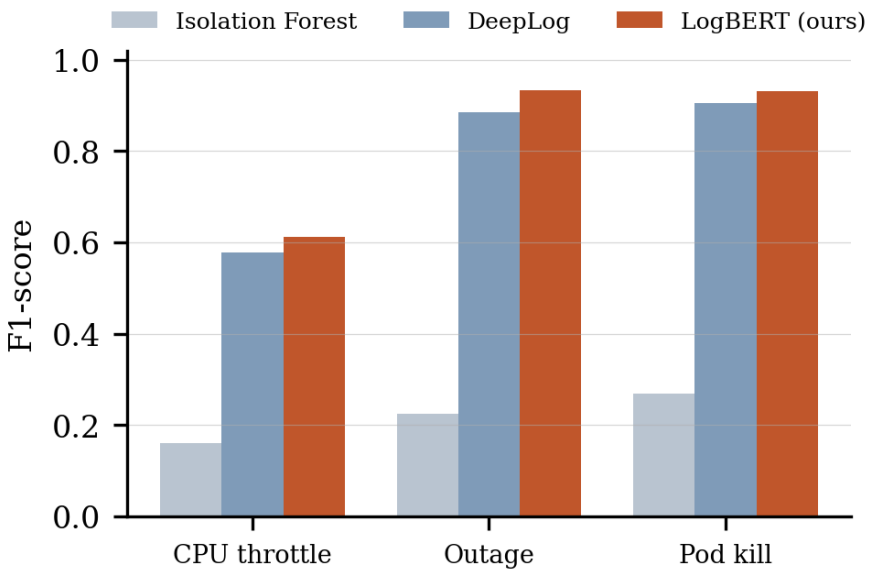


Fig. 2. F1-score per injected fault type, computed on the windows of that fault plus all normal windows.

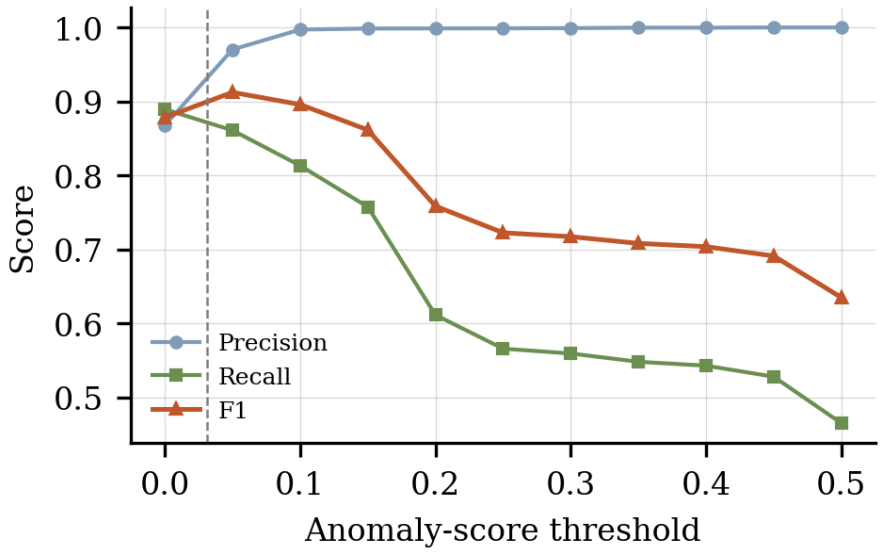


Fig. 3. Effect of the anomaly-score threshold on precision, recall and F1 on the test windows. The dashed line marks the value chosen on the validation run, tau = 0.03.

B. Early warning and recovery

Table IV compares the reactive baseline with the proposed controller on pod kill and CPU throttling, and Fig. 4 shows MTTR per fault type. Lead time is positive when the verdict came before the first failed probe.

TABLE IV. Mitigation outcomes on pod kill and CPU throttling (one run each).

Metric	Reactive	Proactive
Mean time to recovery (s)	pending	pending
Request availability (%)	pending	pending
Failed requests during fault (%)	pending	pending
Episodes with failed requests	pending	pending
Episodes not recovered before the fault ended	pending	pending
Unnecessary actions per hour	pending	pending
Median detection lead time (s)	n/a	pending

Fig. 4 is pending.

Draft — reported values are provisional and pending experimental validation

For the outage fault, reported separately for the reason given in Section IV-C, MTTR is pending s for the reactive baseline and pending s for the controller, with pending% and pending% of requests failing during the fault.

C. Ablation

Table V repeats the detection evaluation with one component changed at a time. The last row applies the two-window persistence rule used by the controller to the full model's window verdicts.

TABLE V. Ablation study on the test windows.

Variant	Precision	Recall	F1
Full model	**0.97**	**0.86**	**0.91**
Without VHM objective	0.99	0.86	0.92
Without service embedding	0.98	0.86	0.92
Encoder with 2 layers	0.98	0.86	0.92
Full model + 2-window persistence rule	0.98	0.84	0.91

D. Threats to validity and limitations

The study uses a single demo application, synthetic faults and a one-node cluster on a laptop; real production logs are noisier, drift with every release and are produced at far higher volume. Only three fault types are covered, injected through the Kubernetes API rather than a chaos tool, so memory leaks and network faults are not evaluated. Each condition is a single run, so the results carry no confidence intervals. Faults are labelled by interval and by the static call graph, which blurs the exact onset and marks some unaffected windows as anomalous. The fault class comes from keyword rules rather than a learned classifier, and the policy has no reschedule or resource-request action. The CPU-throttling fault lives in the running pod, so a restart removes it, which may not hold for a throttle caused by the workload itself. Logs are polled every 2 s, which bounds how early a verdict can arrive and can lose the last lines of a container that crashes between polls. Automatic remediation can also do harm if the detector is wrong, which is why the framework limits its action budget and escalates persistent anomalies to a human. Future work includes more fault types, repeated runs, a learned fault classifier, drift-aware incremental training, evaluation on a public benchmark such as those in Loghub [8], and trials on a multi-node cluster.

VI. CONCLUSION

We presented a closed-loop framework that uses LogBERT anomaly detection to trigger proactive mitigation in Kubernetes. On a single-node Minikube deployment of the Online Boutique application, the detector reaches an F1-score of 0.91 on held-out fault windows (DeepLog: 0.90). Closed-loop recovery results are pending the experimental runs. The approach needs only logs and standard Kubernetes APIs, and the explicit action budget keeps its behaviour predictable.

REFERENCES

[1] H. Guo, S. Yuan and X. Wu, “LogBERT: Log anomaly detection via BERT,” in *Proc. Int. Joint Conf. Neural Networks (IJCNN)*, 2021.

[2] M. Du, F. Li, G. Zheng and V. Srikumar, “DeepLog: Anomaly detection and diagnosis from system logs through deep learning,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2017, pp. 1285–1298.

[3] P. He, J. Zhu, Z. Zheng and M. R. Lyu, “Drain: An online log parsing approach with fixed depth tree,” in *Proc. IEEE Int. Conf. Web Services (ICWS)*, 2017, pp. 33–40.

[4] W. Meng *et al.*, “LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs,” in *Proc. Int. Joint Conf. Artificial Intelligence (IJCAI)*, 2019, pp. 4739–4745.

[5] X. Zhang *et al.*, “Robust log-based anomaly detection on unstable log data,” in *Proc. ACM ESEC/FSE*, 2019, pp. 807–817.

[6] B. Burns, B. Grant, D. Oppenheimer, E. Brewer and J. Wilkes, “Borg, Omega, and Kubernetes,” *ACM Queue*, vol. 14, no. 1, 2016.

[7] Chaos Mesh Authors, “Chaos Mesh: A chaos engineering platform for Kubernetes,” Cloud Native Computing Foundation. [Online]. Available: https://chaos-mesh.org

[8] J. Zhu *et al.*, “Loghub: A large collection of system log datasets for AI-driven log analytics,” in *Proc. IEEE Int. Symp. Software Reliability Engineering (ISSRE)*, 2023.

[9] Google Cloud Platform, “Online Boutique (microservices-demo),” GitHub repository. [Online]. Available: https://github.com/GoogleCloudPlatform/microservices-demo

[10] N. Dragoni *et al.*, “Microservices: Yesterday, today, and tomorrow,” in *Present and Ulterior Software Engineering*, Springer, 2017, pp. 195–216.

[11] F. T. Liu, K. M. Ting and Z.-H. Zhou, “Isolation forest,” in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422.

[12] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.